Project Titan - Distributed Architecture & Systems Specification

System Engineering Architecture Review - Document ID: SPEC-TITAN-2026-REV3 | Cloud Infrastructure Group

1. Project Titan Overview and High-Level Topology

Project Titan represents the next-generation, air-gapped enterprise data plane engineered for high-throughput semantic retrieval, vector embedding orchestration, and autonomous context synthesis across distributed on-premises infrastructure. The architecture is engineered to operate in sovereign environments devoid of public internet egress. The cluster topology comprises an event ingestion fabric, an asynchronous text chunking pipeline, a persistent HNSW vector indexing core, and a local quantized large language model (LLM) inference cluster running on dedicated hardware.

2. Ingestion Fabric and Apache Kafka Event Streaming

Document ingestion events are mediated through a 5-broker Apache Kafka cluster operating with KRaft consensus (removing ZooKeeper dependencies). Ingestion payloads adhere to Apache Avro schemas registered within an internal Confluent-compatible Schema Registry. Topics are partitioned by document_hash and tenant_id with an established replication factor of 3 and min.insync.replicas set to 2. Message buffers are calibrated to support sustained throughput spikes of up to 25,000 document modification events per second with zero payload degradation.

3. Persistent Vector Storage and ChromaDB Index Configuration

The vector indexing tier utilizes a clustered ChromaDB persistent backend configured with Hierarchical Navigable Small World (HNSW) space indices. The distance metric is fixed to Cosine Similarity. Index construction parameters are calibrated at M=32 (bidirectional link density) and efConstruction=200, achieving a balance between index construction time and high-recall graph quality. Search-time exploration is parameterized at efSearch=128, guaranteeing 99.4% recall on top-k=4 nearest neighbor queries across a corpus of 10 million pre-embedded chunks.

4. Relational Metadata, Audit Logging, and TimescaleDB

Document metadata, access control lists (ACLs), user session tokens, and chunk lineage are persisted in a high-availability PostgreSQL 16 cluster equipped with TimescaleDB extensions for query telemetry. Primary and replica nodes replicate synchronously via streaming physical replication with automatic failover managed by Patroni and etcd v3.5. Audit logs record every retrieval request, document chunk hash, and model response timestamp for automated SOC2 compliance auditing.

5. Zero-Trust Service Mesh and Mutual TLS (mTLS)

All inter-service microservice communications transit an internal Istio Service Mesh (version 1.22) enforcing strict mutual TLS (mTLS) with cryptographically validated SPIFFE/SPIRE x509 workload identities. Direct IP-to-IP ingress is blocked by eBPF-driven Cilium network policies. Microservices expose internal gRPC APIs with Protocol Buffers v3, eliminating JSON parsing overhead and reducing inter-node serialization latency to under 1.2 milliseconds.

6. Local Model Inference Engine and Hardware Pinned Accelerators

Inference services execute on dedicated compute nodes hosting NVIDIA A100 (80GB SXM4) or RTX 4090 accelerators. The primary text generation engine runs local Ollama with quantized 'qwen2.5-coder:7b' (Q4_K_M GGUF format),

delivering generation speeds of 72 tokens per second at 32k context lengths. Sentence embedding generation utilizes 'all-MiniLM-L6-v2' (384-dimensional dense vectors) executed locally on CPU via ONNX Runtime with AVX-512 vectorization.

7. Autoscaling Criteria, Health Checks, and Latency SLAs

Workload autoscaling is governed by Kubernetes Horizontal Pod Autoscaler (HPA) using custom Prometheus metrics. Scale-out triggers when average CPU exceeds 70% or queue depth exceeds 250 pending chunks per worker for 30 consecutive seconds. The architectural Service Level Agreement (SLA) mandates: p50 end-to-end query latency < 95ms, p95 < 180ms, and p99 < 350ms, with 99.99% service availability across internal enterprise networks.